Data Structures

Dr. Basma M. Hassan

Faculty of Artificial Intelligence
Kafrelsheikh University

2024/2025

Data Structures

Lecture 1:

What is Data?

- Data
  - A collection of facts from which a conclusion may be drawn
  - Example:
    - Temperature is 35 degrees celcius
    - Conclusion: It is HOT!
- Types of data:
  - Textual: for example, your name (Muhammad)
  - Numeric: for example, your id (090254)
  - Audio: for example, your voice
    - and more

What are Data Structures?

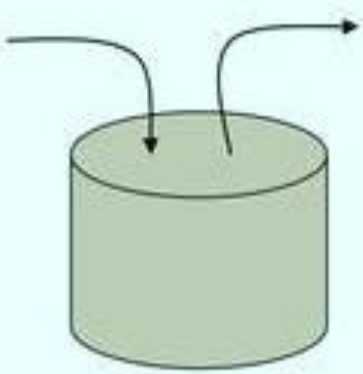
- Data Structure:
  - A particular way of storing and organizing data in a computer so that it can be used efficiently
  - It is a group of data elements grouped together under one name
  - Example:
    - An array of integers

```
int examGrades[30];
```

Types of Data Structures

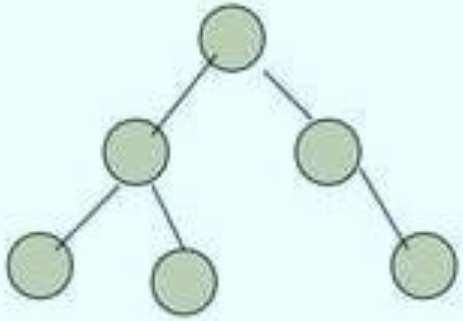
Array

Linked List



Stack

Queue



Tree

- These are just a few of the data structures you will study during your career here at KAU.

Importance of Data Structures

- Goal:
  - We need to organize data
- For what purpose?
  - To facilitate efficient
    - storage of data
    - retrieval of data
    - manipulation of data
- Design Issue:
  - The challenge is to select the most appropriate data structure for the problem
    - Such is one of the motivations for this course

Operations Performed on Data Structures

- Traversing
  - Accessing/visiting each data element exactly once so that certain items in the data may be processed
- Searching
  - Finding the location of a given data element (key) in the structure
- Insertion:
  - Adding a new data element to the structure

Operations Performed on Data Structures

- Deletion
  - Removing a data element from the structure
- Sorting
  - Arrange the data elements in some logical fashion
    - ascending or descending
- Merging
  - Combining data elements form two or more data structures into one

Types of Data Structures

- Based on Memory Allocation:
  - Static (or fixed sized) data structures
    - Such as arrays
  - Dynamic data structures (change size as needed)
    - Such as Linked Lists
      - (coming later in the semester)
- Based on Representation
  - Linear representation
    - Such as arrays and linked lists
  - Non-linear representation
    - Such as trees and graphs

Motivation for Data Structures

- All data structures have a PURPOSE
  - A reason for their creation and motivation
- Think about the one data structure you all know: arrays
- What is the purpose of an array?

Array: Motivation

- You want to store 5 numbers in a program
  - No problem. You define five int variables:

```
int num1, num2, num3, num4, num5;
```

  - Easy enough, right?
  - But what if you want to store 1000 numbers?
    - Are you really going to make 1000 separate variables?

```
int num1, num2,..., num998, num999, num1000;
```

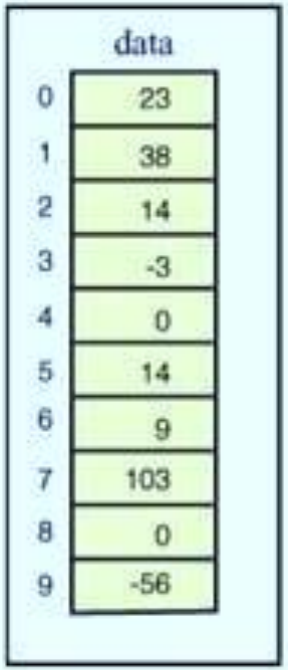
    - That would be CRAZY!
- So what is the solution?
  - A data structure! Specifically, an array!
    - An array is one of the most common data structures.

What is an Array?

- An array is a data structure
  - It is a collection of homogeneous data elements
    - Meaning, all elements are of the same type
    - Examples:
      - An array of student grades
      - An array of student names
      - An array of objects (OOP perspective!)
    - Array elements (or their references) are stored in contiguous/consecutive memory locations
  - Also, an array is a static data structure
    - An array cannot grow or shrink during program execution…the size is fixed

Review of Arrays

- Basic Concepts
  - Array name (data)
  - Index/subscript (0...9)
  - The slots are numbered sequentially starting at zero (Java, C++)
  - If there are N slots in an array, the index will be 0 through N-1
  - Array length = N = 10
  - Array size = N x Size of an element = 40
  - Direct access to an element

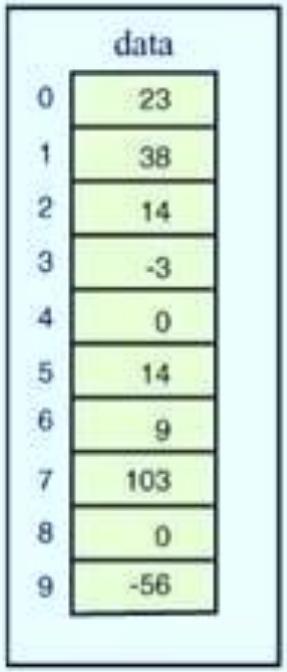


Review of Arrays

- Homogeneity:
  - All elements in an array must have the same data type
- Contiguous memory:
  - Array elements are stored in contiguous memory locations
  - So an array of 100 integers (int is 4 bytes) would be stored in 400 bytes of contiguous memory
    - Each of the 4 byte locations would come right after the previous location in memory

Review of Arrays

- More Concepts:
  - data[-1] always illegal
  - data[10] illegal (10 > upper bound)
  - data[1.5] always illegal
  - data[0] always OK
  - data[9] OK



- Question: What will be the output of?
  - 1.data[5] + 10
  - 2.data[3] = data[3] + 10

Review of Arrays

- Array Dimensionality
  - One dimensional (just a linear list)
  - Example:

5	10	18	30	45	50	60	65	70	80

  - Only one subscript is required to access an individual element

Review of Arrays

- Array Dimensionality
  - Two dimensional (matrix or table)
  - Example: 2x4 matrix (2 rows, 4 columns)

	Col 0	**Col 1**	**Col 2**	**Col 3**
Row 0	20	25	60	40
Row 1	30	15	70	90

Review of Arrays

- 2D Arrays:
  - Given the following array (whose name is 'M')

20	25	60	40
30	15	70	90

  - Two indices/subscripts are now required to reference a given cell
    - You must give a row/column
    - First element is at row 0, column 0
      - M[0][0]
    - What is: M[1][2] ? or M[3][4] ?

Review of Arrays

- Array Operations:
  - Accessing/indexing an element using its index
    - Performance is very fast
    - We can access an index immediately without searching
    - myArray[1250] = 55;
      - we immediately access array spot 1250 of myArray
  - Insertion: add an element at a certain index
    - What if we want to add an element at the beginning?
      - This would be a very slow operation! Why?
        - Because we would have to shift ALL other elements over one position
    - What if we add an element at the end?
      - It would be FAST. Why? No need to shift.

Review of Arrays

- Array Operations:
  - Removal: remove an element at a certain index
    - Remove an element at the beginning of the array
      - Performance is again very slow.
      - Why?
        - Because ALL elements need to shift one position backwards
    - Remove an element at the end of an array
      - Very fast because of no shifting needed
  - Searching through the array:
    - Depends on the algorithm
    - Some algorithms are faster than others
      - More detail coming soon!

Review of Arrays

- Array Declaration:
  - You declare an array as follows:

    ```
    int[] grades;
    ```

  - This simply makes a an array variable (grades)
    - But it does NOT specify where that variable refers to
  - We can also declare the following:

    ```
    int[] grades = new int[10];
    ```

    - Now the array variable grades refers to an array of ten integers
      - By default, numeric elements are initialized to zero

Review of Arrays

- More info on Arrays:
  - When the array is created, memory is reserved for its contents
  - You can also specify the values for the array instead of using the new operator
  - Example:

```
int[] grade = {95, 93, 88}; //array of 3 ints
```

  - To find the length of an array, use the length method

```
int numGrades = grade.length();
```

Review of Arrays

- Arrays' Properties in Java
  - Arrays are objects
  - Arrays are created dynamically, at run time
  - An array type variable holds a reference to the array object
  - An array's length is set when the array is created, and it cannot be changed.
  - Arrays can be duplicated with the `Object.clone()` method
  - An `ArrayIndexOutOfBoundsException` is thrown if index exceeds array length

Review of Arrays

- Array Processing in JAVA
  - java.util.Arrays class provides many built-in functions for sorting and searching
    - The method Arrays.sort(array_name) is used to sort an array
    - The method Arrays.binarySearch(array_name, target) is used to search through a sorted array
    - The method Arrays.equals(array1, array2) is used to check if two arrays are equal
      - Meaning, if the same values are at each index position